ANS w Elblągu

Instytut Informatyki Stosowanej

Im. Krzysztofa Brzeskiego

Przetwarzanie równoległe i rozproszone

Studium stacjonarne, sem 4, 2025/2026

Projekt Semestralny

Autorzy:

- Adrian Czarnota 21442
- Dawid Rzepka 21299

Grupa: 2

Data: 14.04.2026

Repozytorium GitHub: https://github.com/rzepeeek/Projekt_Python_Scraping

Cel projektu

Celem projektu było stworzenie aplikacji do pobierania, przetwarzania oraz przechowywania danych (np. Adresy email, linki, numery telefonów, adresy) w architekturze rozproszonej. Użytkownik, korzystając z interfejsu webowego, wprowadza adresy URL, które następnie są przetwarzane przez aplikację. System pobiera dane w sposób asynchroniczny, analizuje je i zapisuje w bazie danych MongoDB.

Projekt miał na celu praktyczne wykorzystanie technologii backendowych oraz konteneryzacji. Kluczowym założeniem było również rozdzielenie aplikacji na niezależne moduły działające jako osobne kontenery Docker, takie jak interfejs użytkownika, silnik przetwarzający oraz baza danych.

Opis działania

Użytkownik wprowadza adresy URL poprzez interfejs webowy, po czym dane są przesyłane do silnika scrapującego. Silnik pobiera zawartość stron internetowych w

sposób asynchroniczny (asyncio), a następnie przetwarza dane równoległe przy użyciu multiprocessing. Zebrane dane są analizowane (parsowane) przy użyciu biblioteki BeautifulShop, a następnie zapisywane w bazie danych MongoDB.

Aplikacja działa w architekturze rozproszonej, gdzie poszczególne komponenty komunikują się ze sobą poprzez interfejs sieciowy.

Struktura systemu

System składa się z trzech głównych modułów:

- Interface (Flask + HTML/CSS) - odpowiada za przyjmowanie adresów URL oraz komunikację z użytkownikiem.
- Engine (Silnik scrapujący) - realizuje pobieranie, przetwarzanie i analizę danych.
- Database (MongoDB) - przechowuje dane pozyskane ze stron.

Wszystko uruchamianie jest jako kontenery za pomocą Docker Compose.

Technologie użyte w projekcie

W projekcie wykorzystano następujące technologie:

- Python 3.14 - język programowania
- Flask – framework webowy (interfejs użytkownika)
- asyncio – asynchroniczne operacje sieciowe
- multiprocessing – przetwarzanie równoległe
- BeautifulSoup – parsowanie HTML
- MongoDB – baza danych NoSQL
- Docker – konteneryzacja aplikacji
- HTML + CSS – interfejs użytkownika

Instalacja i uruchomienie

Uruchomienie przy użyciu Docker'a:

Na początku aby uruchomić projekt, trzeba sklonować repozytorium:

```
git clone https://github.com/rzepeeek/Projekt_Python_Scraping.git
cd Projekt_Python_Scraping/WEB-SCRAPER-PROJECT
```

Następnie można uruchomić kontenery za pomocą polecenia:

```
docker-compose up –build
```

Po uruchomieniu aplikacja jest dostępna pod adresem:

http://localhost:5000/

Uruchomienie wersji lokalnej:

1. Klonowanie repozytorium:

```
git clone https://github.com/rzepeeek/Projekt_Python_Scraping.git
cd Projekt_Python_Scraping/WEB-SCRAPER-PROJECT
```

2. Utworzenie środowiska:

```
python -m venv .venv
.venv\Scripts\activate
```

3. Instalacja zależności:

```
pip install -r requirements.txt
```

4. Uruchomienie MongoDB:

```
mongod --dbpath C:\data\db
```

5. Uruchomienie aplikacji:

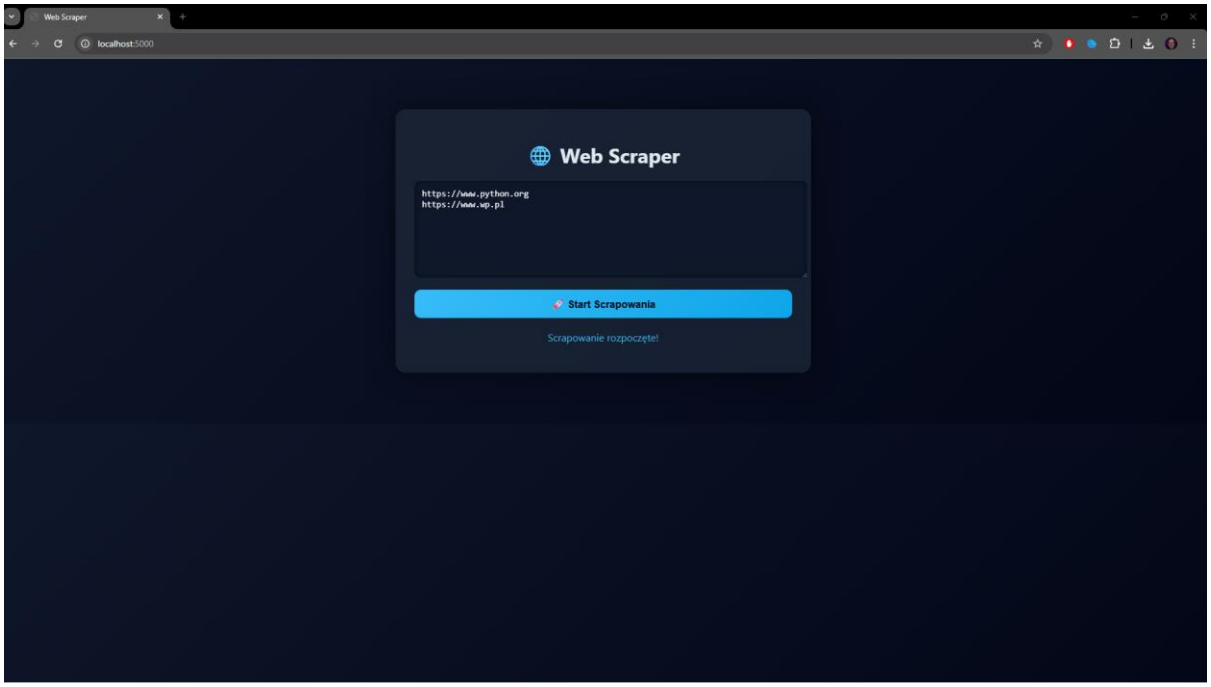
```
cd engine
python main.py

cd interface
python app.py
```

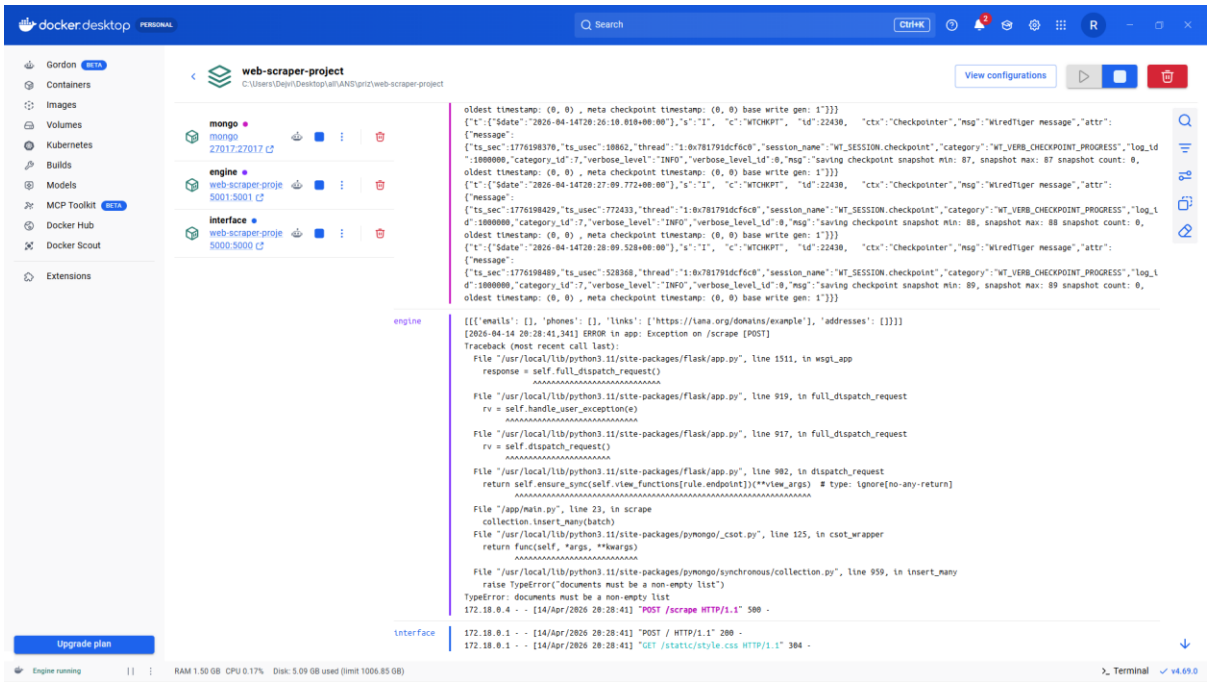
6. Otwórz:

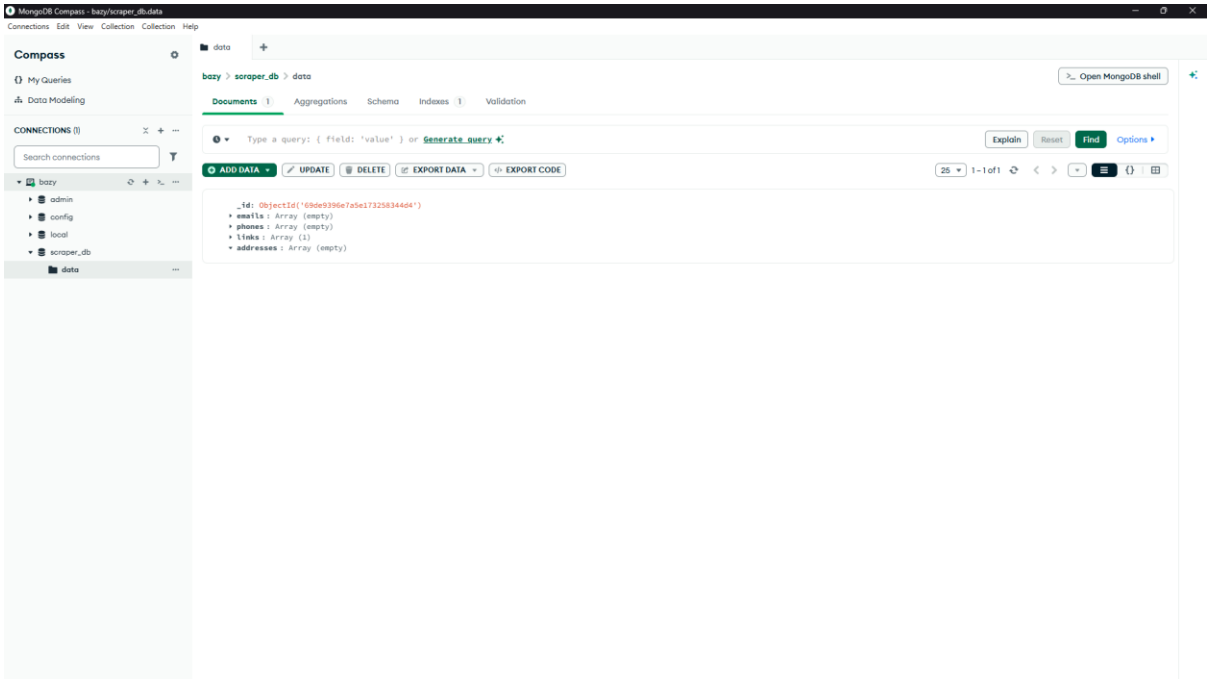
http://localhost:5000/

Po odpaleniu strony pojawi się formularz, do którego można wkleić adres URL. Po kliknięciu przycisku “Start Scrapowania” rozpocznie się pobieranie danych, a komunikat “Scrapowanie rozpoczęte” poinformuje o poprawnym rozpoczęciu działania silnika.

Dane będą zapisywane do bazy danych MongoDB w scraper_db -> data.




docker desktop
PERSONAL
Search
Ctrl+K
Gordon BETA
Containers
Images
Volumes
Kubernetes
Builds
Models
MCP Toolkit BETA
Docker Hub
Docker Scout
Extensions
Upgrade plan
web-scraper-project
C:\Users\Dejvi\Desktop\all\ANS\priz\web-scraper-project
View configurations
mongo
mongo
27017:27017
engine
web-scraper-proje
5001:5001
interface
web-scraper-proje
5000:5000
oldest timestamp: (0, 0) , meta checkpoint timestamp: (0, 0) base write gen: 1"}}}
{"t":{"$date":"2026-04-14T20:26:10.010+00:00"},"s":"I", "c":"WTCHKPT", "id":22430, "ctx":"Checkpointer","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1776198370,"ts_usec":10862,"thread":"1:0x781791dcf6c0","session_name":"WT_SESSION.checkpoint","category":"WT_VERB_CHECKPOINT_PROGRESS","log_id":1000000,"category_id":7,"verbose_level":"INFO","verbose_level_id":0,"msg":"saving checkpoint snapshot min: 87, snapshot max: 87 snapshot count: 0,
oldest timestamp: (0, 0) , meta checkpoint timestamp: (0, 0) base write gen: 1"}}}
{"t":{"$date":"2026-04-14T20:27:09.772+00:00"},"s":"I", "c":"WTCHKPT", "id":22430, "ctx":"Checkpointer","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1776198429,"ts_usec":772433,"thread":"1:0x781791dcf6c0","session_name":"WT_SESSION.checkpoint","category":"WT_VERB_CHECKPOINT_PROGRESS","log_id":1000000,"category_id":7,"verbose_level":"INFO","verbose_level_id":0,"msg":"saving checkpoint snapshot min: 88, snapshot max: 88 snapshot count: 0,
oldest timestamp: (0, 0) , meta checkpoint timestamp: (0, 0) base write gen: 1"}}}
{"t":{"$date":"2026-04-14T20:28:09.528+00:00"},"s":"I", "c":"WTCHKPT", "id":22430, "ctx":"Checkpointer","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1776198489,"ts_usec":528368,"thread":"1:0x781791dcf6c0","session_name":"WT_SESSION.checkpoint","category":"WT_VERB_CHECKPOINT_PROGRESS","log_id":1000000,"category_id":7,"verbose_level":"INFO","verbose_level_id":0,"msg":"saving checkpoint snapshot min: 89, snapshot max: 89 snapshot count: 0,
oldest timestamp: (0, 0) , meta checkpoint timestamp: (0, 0) base write gen: 1"}}}
engine
[[{'emails': [], 'phones': [], 'links': ['https://lana.org/domains/example'], 'addresses': []}]]
[2026-04-14 20:28:41,341] ERROR in app: Exception on /scrape [POST]
Traceback (most recent call last):
File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 1511, in wsgi_app
response = self.full_dispatch_request()
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 919, in full_dispatch_request
rv = self.handle_user_exception(e)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 917, in full_dispatch_request
rv = self.dispatch_request()
^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 902, in dispatch_request
return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
^^^^^^^^^^^^^^^^^^^



Architektura aplikacji

Aplikacja działa w architekturze rozproszonej opartej na mikroserwiasach.

[INTERFACE] -> [ENGINE] -> [MONGO DB]

- Każdy moduł działa jako osobna usługa
- Komunikacja odbywa się przez HTTP
- W Dockerze każdy moduł działa w osobnym kontenerze

Zaletami tego jest:

- Skalowalność
- Separacja odpowiedzialności
- Łatwość rozbudowy

Testowanie

Testowanie aplikacji obejmowało:

- Sprawdzenie poprawności scrapowania różnych stron
- Testy zapisu danych do MongoDB
- Testy komunikacji między modułami
- Testy działania w środowisku Docker

Przykładowe strony testowe:

- http://example.com

- https://python.org
- https://www.wp.pl

Wyniki:

- Poprawne pobieranie danych
- Poprawny zapis do bazy
- Stablina praca aplikacji

Wnioski

- Zastosowanie asyncio znacząco przyspiesza pobieranie danych
- multiprocessing umożliwia wykorzystanie wielu rdzeni CPU
- MongoDB dobrze sprawdza się przy danych niestrukturalnych
- Docker upraszcza uruchamianie i wdrażanie systemu

Projekt spełnia wszystkie założenia i działa stabilnie.

Dalszy rozwój

Możliwe kierunki rozwoju:

- Dodanie analizy treści stron
- Wykrywanie danych kontaktowych bardziej zaawansowanymi metodami
- Dodanie panelu administracyjnego
- Obsługa kolejek

Dodatkowe funkcjonalności

Możliwość skalowania

System może zostać rozszerzony poprzez:

- Zwiększenie liczby procesów
- Uruchamianie wielu instancji silników
- Wdrożenie w środowisku chmurowym

Modularność i łatwa rozbudowa

Dzięki podziałowi na moduły:

- Można łatwo wymienić bazy danych
- Można dodać nowe typy danych

- Można rozszerzyć frontend bez zmian backendu

PODSUMOWANIE

Projekt stanowi kompletny system do scrapowania danych z Internetu, spełniający wszystkie wymagania zadania.

Zastosowane technologie zapewniają wydajność, skalowalność oraz możliwość dalszego rozwoju.